PREDICTIVE MAINTENANCE FOR INDUSTRIAL MACHINES

Machine Failure Prediction using Machine Learning

Project Report

Based on the supplied Jupyter Notebook and AI4I 2020 dataset

Abstract

This project applies supervised machine-learning and anomaly-detection methods to predict machine failures using the AI4I 2020 predictive-maintenance dataset. The notebook explores the data, prepares six input features, compares multiple classifiers, tunes a Random Forest model, evaluates alert thresholds using an assumed cost model, and demonstrates saving and reloading the trained model. The dataset contains 10,000 records, and the target is imbalanced: 3.39% of records indicate machine failure. Results therefore emphasize precision, recall, F1-score and PR-AUC alongside accuracy.

1. Introduction

Unexpected machine failures can interrupt production, increase maintenance costs and reduce equipment availability. Predictive maintenance uses equipment measurements and historical outcomes to estimate the likelihood of failure so that maintenance can be planned before an unexpected breakdown. This project treats machine failure prediction as a binary classification task: 0 represents no machine failure and 1 represents machine failure.

2. Project Objectives

The objectives implemented in the notebook are to inspect and understand the dataset; identify missing values and target imbalance; prepare suitable input features; train and compare machine-learning models; tune a Random Forest classifier using cross-validation; examine the effect of changing the probability threshold; and save the selected model and scaler for later prediction.

3. Dataset Description

The supplied CSV is the AI4I 2020 dataset. The notebook loads it with pandas. It contains 10,000 rows and 14 columns. The columns include identifiers (UDI and Product ID), product type, operating measurements (air temperature, process temperature, rotational speed, torque and tool wear), the target label (Machine failure), and five failure-mode indicators (TWF, HDF, PWF, OSF and RNF). All 14 columns contain 10,000 non-null values in the notebook output.

Column group	Fields	Role in notebook
Identifiers	UDI, Product ID	Removed from model inputs
Product category	Type	Encoded L=0, M=1, H=2
Operating measurements	Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min]	Predictor features
Target	Machine failure	Binary target (0/1)
Failure-mode indicators	TWF, HDF, PWF, OSF, RNF	Excluded from predictors to avoid target leakage

4. Exploratory Data Analysis (EDA)

The notebook first inspects the first records, data types and descriptive statistics. It then checks null counts and visualizes the target distribution and failure-mode counts. No missing values were found in any column. The target distribution is strongly imbalanced: 9,661 records (96.61%) are non-failures and 339 records (3.39%) are failures.

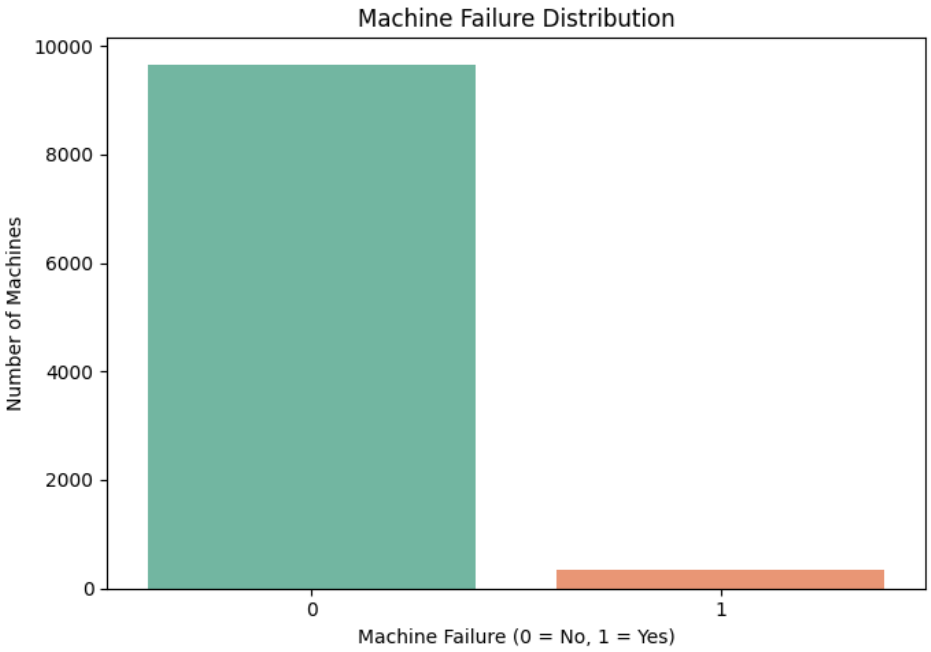


Figure 1. Original notebook chart: machine failure distribution.

The imbalance means a model could achieve high accuracy by predicting the majority class while missing many actual failures. For that reason, the notebook also reports recall, precision, F1-score and PR-AUC rather than judging models by accuracy alone.

4. Exploratory Data Analysis (continued)

The notebook counts the five failure-mode indicator columns. These counts are TWF=46, HDF=115, PWF=95, OSF=98 and RNF=19. These are separate indicator totals; they should not be interpreted as mutually exclusive categories or summed to obtain the number of failed machines.

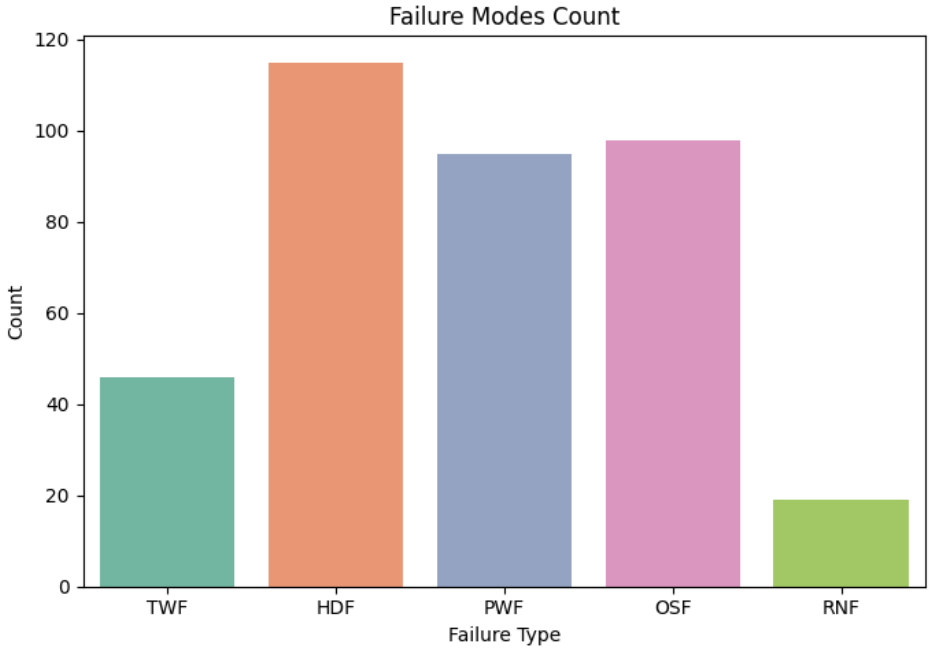


Figure 2. Original notebook chart: failure-mode counts.

5. Data Preprocessing and Feature Selection

The Type column is converted from text to numeric values using the mapping L=0, M=1 and H=2. The identifier columns UDI and Product ID are removed because they are not used as predictive measurements. The Machine failure column is separated as the target. The five failure-mode indicator columns are also removed from X to avoid target leakage because they describe failure modes associated with the target rather than independent operating measurements.

The final feature set contains six columns: Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm] and Tool wear [min]. The data is split into training and test sets using an 80:20 ratio, random_state=42 and stratification on the target. This produces 8,000 training records and 2,000 test records. StandardScaler is fitted on the training features and then applied to both training and test features.

6. Models Implemented

Model	Approach in notebook
Logistic Regression	Baseline linear classifier
Decision Tree	Tree-based classifier
Random Forest	Ensemble of decision trees
Gradient Boosting	Sequential boosting ensemble
K-Nearest Neighbors (KNN)	Distance-based classifier
Isolation Forest	Anomaly-detection method; anomalies mapped to failure

The five supervised classifiers are trained on the scaled training data and evaluated on the same held-out test set. Isolation Forest is fitted separately as an anomaly-detection comparison. The notebook records accuracy, recall, precision, F1-score, PR-AUC, missed failures (false negatives) and false alarms (false positives).

7. Model Comparison Results

The following table reproduces the model-comparison values printed by the notebook. Scores are reported on the held-out test set.

Model	Accuracy	Precision	Recall	F1	PR-AUC
Logistic Regression	0.9675	0.6364	0.1029	0.1772	0.4238
Decision Tree	0.9775	0.6667	0.6765	0.6715	0.4620
Random Forest	0.9840	0.8750	0.6176	0.7241	0.8135
Gradient Boosting	0.9865	0.8868	0.6912	0.7769	0.8126
KNN	0.9740	0.8333	0.2941	0.4348	0.4586
Isolation Forest	0.9435	0.1429	0.1324	0.1374	0.1208

Interpretation: Gradient Boosting has the highest accuracy (98.65%), recall (69.12%) and F1-score (0.7769) among the compared models. Random Forest has the highest PR-AUC (0.8135), slightly above Gradient Boosting (0.8126), and precision of 87.50%. Logistic Regression has high accuracy but very low recall (10.29%), showing why accuracy alone is insufficient for this imbalanced problem. These results do not establish one universally best model; the choice depends on the relative importance of missed failures and false alarms.

8. Random Forest Hyperparameter Tuning

The notebook applies GridSearchCV to RandomForestClassifier(random_state=42), using three-fold cross-validation and F1 as the scoring metric. The parameter grid tests n_estimators values of 50 and 100, and max_depth values of 5, 10 and None. The best parameters printed are max_depth=None and n_estimators=100.

The tuned estimator is evaluated on the test set. Its confusion matrix is shown below. Rows represent actual classes and columns represent predicted classes.

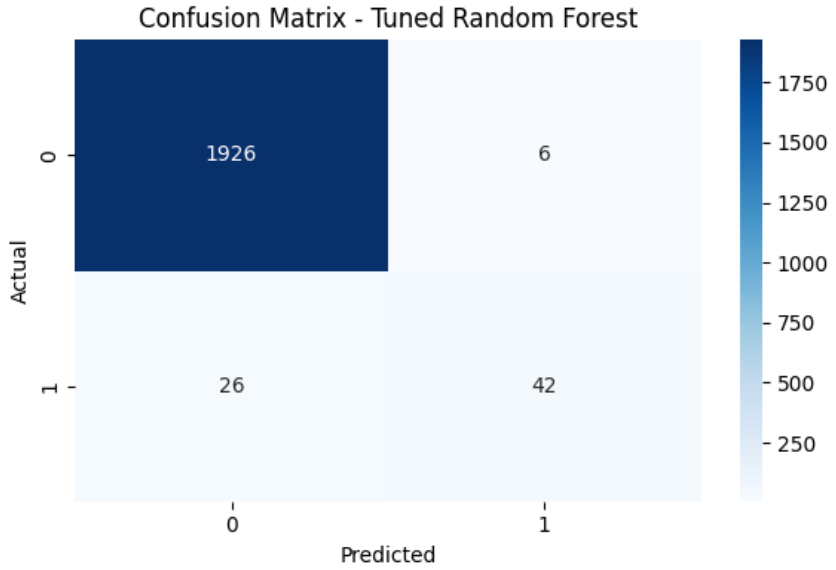


Figure 3. Original notebook confusion-matrix chart for the tuned Random Forest.

The matrix values are TN=1,926, FP=6, FN=26 and TP=42. Thus, the model correctly identifies 42 failure cases but misses 26 actual failures at the default 0.5 classification threshold. Its reported comparison metrics are accuracy 98.40%, precision 87.50%, recall 61.76% and F1-score 0.7241.

9. Probability Threshold and Cost Analysis

The notebook examines thresholds from 0.1 to 0.7 using predicted failure probabilities from the tuned Random Forest. It assumes a cost of 500 units for each missed failure (FN) and 50 units for each false alarm (FP). These are illustrative assumptions used by the notebook, not verified real-world factory costs.

Threshold	Recall	Precision	FN	FP	Assumed total cost
0.1	0.8824	0.3974	8	91	8,550
0.2	0.8382	0.5644	11	44	7,700
0.3	0.7794	0.6974	15	23	8,650
0.4	0.7353	0.8333	18	10	9,500
0.5	0.6176	0.8750	26	6	13,300
0.6	0.5588	0.8837	30	5	15,250
0.7	0.3971	1.0000	41	0	20,500

Among the thresholds tested, 0.2 gives the lowest assumed total cost (7,700). It detects more failures than the default 0.5 threshold (recall 83.82% versus 61.76%) but also produces more false alarms (44 versus 6). This illustrates the trade-off between catching failures and triggering unnecessary inspections. A real deployment would need validated operational costs and validation on data representative of future machines.

10. Saving and Testing the Model

The notebook saves the tuned Random Forest estimator as best_rf_model.pkl and the fitted StandardScaler as scaler.pkl using joblib. It then reloads both files and tests one illustrative input row with feature order [Type, Air temperature, Process temperature, Rotational speed, Torque, Tool wear]: [0, 300.0, 310.0, 1400, 60.0, 200]. The notebook reports a predicted class of 1 (failure) and a predicted failure probability of 55.0%. This is a demonstration sample, not evidence of live deployment performance.

11. Conclusion

This project demonstrates an end-to-end machine-learning workflow for machine failure prediction: data inspection, EDA, feature preparation, stratified splitting, scaling, model comparison, hyperparameter tuning, threshold analysis and model persistence. The EDA revealed a strongly imbalanced target and no missing values. Gradient Boosting achieved the strongest F1-score and accuracy in the model comparison, while Random Forest achieved the highest PR-AUC. The threshold analysis showed that lowering the decision threshold can improve failure recall at the cost of more false alarms. For practical use, the model should be validated on new time-separated or production data, and the alert threshold should be selected with domain-approved cost estimates.

12. Limitations and Future Work

The notebook uses a single train-test split and does not report an independent external validation set. The cost values in threshold analysis are assumptions. The notebook does not provide a deployment pipeline, live monitoring, model-drift checks or a maintenance feedback loop. Future work could evaluate time-based validation, calibration, class-imbalance techniques, additional operating data, and threshold selection with input from maintenance engineers.

13. References

1. AI4I 2020 Predictive Maintenance Dataset (the CSV supplied with this project).
2. Scikit-learn documentation for classification models, preprocessing, model selection and evaluation: https://scikit-learn.org/stable/.
3. pandas documentation: https://pandas.pydata.org/docs/.
4. Matplotlib documentation: https://matplotlib.org/stable/.
5. Seaborn documentation: https://seaborn.pydata.org/.

Report preparation note: Dataset dimensions, feature handling, model scores, threshold results and sample prediction are transcribed from the supplied notebook outputs. The three EDA/evaluation images in this report are extracted from the original notebook outputs.